Regresiones lineales

M4: Análisis exploratorio de datos

|**AE4**: Implementar un modelo de regresión lineal para el análisis de la dependencia de variables utilizando lenguaje python.

alkemy

Introducción👩‍💻

La visualización de datos es una herramienta esencial en el análisis estadístico, ya que permite interpretar información compleja de forma intuitiva y efectiva. Cuando se combina con modelos estadísticos como la regresión lineal, su utilidad se multiplica, ya que facilita la comprensión de patrones, tendencias y relaciones entre variables. Una de las bibliotecas más potentes para este propósito en el ecosistema Python es **Seaborn**, que ofrece una gran variedad de funciones para realizar análisis visuales detallados.

Este manual se centra en la aplicación de **regresiones lineales simples y múltiples**, junto al uso de Seaborn para la representación de estas relaciones. Además, se explorarán los supuestos que deben cumplirse en un modelo lineal, el uso de la librería **Statsmodels** para ajustar e interpretar modelos, y el análisis de métricas de evaluación como el R2, el MSE y el MAE.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Comprender el concepto de regresión lineal y su utilidad en el análisis de datos.
- Identificar y aplicar los supuestos que deben cumplirse en un modelo de regresión.
- Implementar modelos de regresión lineal simple y múltiple utilizando la librería Statsmodels.
- Representar gráficamente modelos de regresión utilizando la librería Seaborn.
- Evaluar el desempeño de modelos regresivos mediante indicadores estadísticos como R2 y MSE.

¿Qué es una regresión lineal simple?

La regresión lineal simple es un método estadístico que permite modelar la relación entre una variable independiente (predictora) y una variable dependiente (respuesta) mediante una línea recta. Esta técnica busca encontrar la mejor recta que se ajuste a los datos observados, con el objetivo de predecir el valor de la variable respuesta a partir de un nuevo valor de la variable predictora.

La ecuación de la regresión lineal simple se expresa como: , donde es el intercepto (ordenada al origen), es el coeficiente de la pendiente y es el término de error aleatorio. Esta fórmula permite hacer inferencias sobre el comportamiento de la variable dependiente según los cambios en la independiente.

Para representar visualmente una regresión lineal simple, Seaborn ofrece funciones como `sns.lmplot()` o `sns.regplot()`. Por ejemplo:

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.regplot(x="total_bill", y="tip", data=sns.load_dataset("tips"))
plt.title("Relación entre total de cuenta y propina")
plt.show()
```

Este código genera un scatterplot con la línea de regresión ajustada, permitiendo observar visualmente la relación entre las dos variables.

Complementariamente, la regresión puede ser ajustada de forma estadística utilizando la librería `statsmodels`. Esto permite obtener los coeficientes y , junto con otras estadísticas relevantes. Un ejemplo simple:

```
import statsmodels.api as sm
import pandas as pd

data = sns.load_dataset("tips")
X = sm.add_constant(data["total_bill"])
y = data["tip"]
modelo = sm.OLS(y, X).fit()
print(modelo.summary())
```

Este resumen incluye valores de R2, p-valores, errores estándar y otros parámetros que permiten evaluar la calidad del modelo.

Determinación de los coeficientes de regresión

Los coeficientes de regresión representan los parámetros del modelo que describen la relación entre la variable independiente y la variable dependiente. En el caso de la regresión lineal simple, el modelo tiene dos coeficientes principales: el intercepto , que indica el valor esperado de la variable dependiente cuando la variable independiente es cero, y la pendiente , que indica cuánto cambia la variable dependiente por cada unidad de cambio en la independiente.

Estos coeficientes se calculan utilizando el método de los mínimos cuadrados ordinarios (OLS), que minimiza la suma de los cuadrados de las diferencias entre los valores observados y los valores predichos por el modelo. Esta técnica garantiza que la recta ajustada sea la que mejor se adapta a los datos disponibles, en términos de error cuadrático medio.

Con la librería `statsmodels`, podemos obtener los coeficientes de forma sencilla. Usando el modelo ajustado en la sección anterior, el método `.params` devuelve los valores de y . Además, el método `.summary()` proporciona información detallada, incluyendo errores estándar, t-estadísticos y p-valores, que son útiles para evaluar la significancia de cada coeficiente:

```
print(modelo.params)
```

Este comando imprimirá algo como:

```
const         0.920270
total_bill    0.105025
dtype: float64
```

La interpretación de estos resultados es fundamental: el coeficiente de `total_bill` indica que por cada unidad monetaria adicional en la cuenta total, se espera que la propina aumente en promedio 0.105 unidades monetarias, manteniendo las demás condiciones constantes. El valor del intercepto sugiere cuál sería la propina estimada si la cuenta fuese cero, aunque en muchos contextos, este valor no tiene un significado práctico y solo sirve como parte del ajuste estadístico del modelo.

Métricas de error y el indicador de ajuste R2

La evaluación de un modelo de regresión no se limita a observar su ecuación o sus coeficientes; es fundamental contar con métricas que midan su precisión y

capacidad predictiva. Las métricas de error nos indican cuánto se desvían las predicciones del modelo respecto a los valores reales. Las más comunes son el **Error Absoluto Medio (MAE)**, el **Error Cuadrático Medio (MSE)** y la **Raíz del Error Cuadrático Medio (RMSE)**. Estas métricas permiten cuantificar el desempeño de un modelo y compararlo con otras alternativas.

El MAE calcula el promedio de las diferencias absolutas entre los valores observados y los valores predichos. El MSE, por su parte, eleva al cuadrado estas diferencias antes de promediarlas, lo que penaliza más fuertemente los errores grandes. Finalmente, el RMSE es simplemente la raíz cuadrada del MSE, lo que lo convierte en una métrica en las mismas unidades que la variable dependiente, facilitando su interpretación.

Otra métrica clave es el **coeficiente de determinación R2**, que indica qué proporción de la variabilidad de la variable dependiente puede explicarse por el modelo. Su valor va de 0 a 1, donde un valor cercano a 1 indica un buen ajuste. En Python, estas métricas pueden calcularse con `sklearn.metrics`:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

predicciones = modelo.predict(X)
print("MAE:", mean_absolute_error(y, predicciones))
print("MSE:", mean_squared_error(y, predicciones))
print("RMSE:", mean_squared_error(y, predicciones, squared=False))
print("R2:", r2_score(y, predicciones))
```

En la práctica, estas métricas permiten identificar si un modelo es útil o si necesita ajustes. Por ejemplo, un MSE muy alto puede sugerir outliers o que el modelo no representa bien la relación entre variables. El R2 también puede disminuir si se agregan variables irrelevantes en regresiones múltiples, lo que resalta la importancia de seleccionar bien los predictores. Al evaluar un modelo, siempre es recomendable combinar estas métricas con la inspección visual y la validación cruzada.

Implementación en Python con librería Statsmodels

La implementación de modelos de regresión en Python puede realizarse de forma eficiente y detallada utilizando la librería `statsmodels`, que permite ajustar modelos estadísticos y obtener informes completos con métricas clave. En esta sección, exploraremos cómo construir un modelo paso a paso utilizando datos reales, desde la importación hasta la interpretación de resultados.

Para comenzar, se debe importar la base de datos y preparar las variables. En este ejemplo, se utilizará el dataset `"tips"` disponible en Seaborn, donde se buscará predecir el monto de la propina (`tip`) en función del total de la cuenta (`total_bill`). La variable dependiente será `tip` y la independiente será `total_bill`. A continuación, se agrega una constante a la matriz de variables predictoras y se ajusta el modelo:

```
import statsmodels.api as sm
import seaborn as sns

data = sns.load_dataset("tips")
X = sm.add_constant(data["total_bill"])
y = data["tip"]
modelo = sm.OLS(y, X).fit()
```

Una vez ajustado el modelo, se puede acceder a un resumen estadístico completo utilizando `.summary()`:

```
print(modelo.summary())
```

Este reporte incluye coeficientes estimados, intervalos de confianza, estadísticos t, p-valores, R2 y R2 ajustado, entre otros. Esta información es esencial para interpretar la relación entre variables y determinar si los coeficientes son estadísticamente significativos.

Finalmente, el modelo puede utilizarse para hacer predicciones con nuevos datos. Por ejemplo, si queremos estimar la propina para una cuenta de $30:

```
nueva_cuenta = sm.add_constant(pd.DataFrame({"total_bill": [30]}))
prediccion = modelo.predict(nueva_cuenta)
print(prediccion)
```

Este enfoque permite no solo ajustar y entender un modelo de regresión, sino también comunicar sus resultados de forma clara, apoyándose en visualizaciones complementarias con `Seaborn` para validar visualmente los supuestos del modelo y su ajuste general.

Regresión lineal múltiple: definición y aplicación

La regresión lineal múltiple es una extensión del modelo lineal simple que permite incluir más de una variable predictora en el análisis. Su objetivo es modelar la relación entre una variable dependiente y varias independientes para mejorar la capacidad predictiva y captar relaciones más complejas. La fórmula general del modelo es:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n + \varepsilon$$

Cada coeficiente βi\beta_iβi representa el efecto de su variable asociada xix_ixi, manteniendo constantes las demás.

Este tipo de análisis es especialmente útil cuando se sospecha que múltiples factores influyen sobre una variable. Por ejemplo, si queremos predecir el monto de una propina, podríamos considerar variables como el total de la cuenta, el tamaño del grupo, el día de la semana y si el cliente fue atendido por una moza mujer o no. Incluir múltiples predictores permite una modelización más precisa.

En Python, la implementación de una regresión múltiple con `statsmodels` es muy similar a la regresión simple. Basta con incluir múltiples columnas como predictores en el DataFrame:

```
X_multi = sm.add_constant(data[["total_bill", "size"]])
y = data["tip"]
modelo_multi = sm.OLS(y, X_multi).fit()
print(modelo_multi.summary())
```

La salida nos dará los coeficientes de cada variable, su significancia estadística, el R2 y el R2 ajustado, entre otras métricas. También es útil representar gráficamente las relaciones parciales para analizar si existen tendencias lineales entre cada predictor y la variable respuesta. Aunque Seaborn no permite visualizar directamente modelos múltiples en un solo gráfico, podemos usar `pairplot()` o `lmplot()` de forma desagregada para explorar cada relación individual:

```
sns.lmplot(x="size", y="tip", data=data)
```

Este gráfico permite ver si una variable específica (por ejemplo, el tamaño del grupo) tiene una relación lineal con la propina, lo cual ayuda a validar la inclusión de ese predictor en el modelo múltiple.

Supuestos de una regresión múltiple

Para que un modelo de regresión múltiple sea válido y útil, debe cumplir ciertos supuestos estadísticos. Estos supuestos permiten que las estimaciones de los coeficientes sean consistentes, insesgadas y eficientes. Los principales supuestos son: **linealidad**, **independencia de los errores**, **homocedasticidad**, **normalidad del error**, y **ausencia de multicolinealidad** entre los predictores.

El supuesto de **linealidad** establece que la relación entre cada predictor y la variable dependiente debe ser lineal. Este supuesto puede verificarse visualmente mediante gráficos de dispersión entre cada variable independiente y la dependiente. Si las relaciones no son lineales, es posible que se requieran transformaciones de las variables (como logaritmos) o el uso de modelos no lineales.

El supuesto de **homocedasticidad** implica que la varianza del error es constante a lo largo de todos los valores de las variables independientes. Para evaluarlo, se puede graficar los residuos del modelo frente a los valores predichos:

```
import matplotlib.pyplot as plt

residuos = modelo_multi.resid
fitted = modelo_multi.fittedvalues
plt.scatter(fitted, residuos)
plt.axhline(0, color="red")
plt.xlabel("Valores predichos")
plt.ylabel("Residuos")
plt.title("Gráfico de residuos vs valores predichos")
plt.show()
```

Si los residuos muestran un patrón aleatorio en torno a la línea 0, el supuesto está cumplido. Si, en cambio, se observa un patrón en forma de embudo o curva, podría estar presente heterocedasticidad.

La **normalidad del error** se refiere a que los residuos del modelo deben seguir una distribución normal. Esto se puede comprobar con un QQ-plot:

```
import scipy.stats as stats
import matplotlib.pyplot as plt

stats.probplot(residuos, dist="norm", plot=plt)
plt.title("QQ-Plot de residuos")
plt.show()
```

Si los puntos se alinean aproximadamente sobre la línea diagonal, el supuesto se cumple. De lo contrario, puede ser necesario aplicar transformaciones o revisar la presencia de valores atípicos.

Finalmente, la **ausencia de multicolinealidad** se asegura revisando que las variables independientes no estén altamente correlacionadas entre sí. Esto puede evaluarse con el **factor de inflación de la varianza (VIF)**. Un VIF superior a 5 o 10 puede indicar problemas:

```python
from statsmodels.stats.outliers_influence import variance_inflation_factor

vif_data = pd.DataFrame()
vif_data["feature"] = X_multi.columns
vif_data["VIF"] = [variance_inflation_factor(X_multi.values, i) for i in range(X_multi.shape[1])]
print(vif_data)
```

El cumplimiento de estos supuestos es clave para asegurar la validez de las inferencias que se extraen del modelo. Ignorarlos puede conducir a conclusiones erróneas o predicciones poco confiables.

Indicador R^2 ajustado y selección del modelo

El coeficiente de determinación R2R^2R2 mide la proporción de la varianza en la variable dependiente que puede ser explicada por el modelo. Aunque esta métrica es útil para evaluar el ajuste general, tiene una limitación importante: **siempre aumenta al agregar más variables al modelo**, incluso si estas variables no son relevantes. Es por eso que, en contextos de regresión múltiple, se prefiere utilizar el **R^2 ajustado**.

El R2R^2R2 ajustado penaliza la inclusión de predictores innecesarios, ajustando su valor según la cantidad de variables y observaciones. Esto lo convierte en una métrica más realista al comparar modelos con distinto número de variables. En Python, al utilizar `.summary()` de `statsmodels`, el valor de `Adj. R-squared` ya aparece junto al R^2 tradicional.

Ejemplo:

```python
print(modelo_multi.rsquared)        # R²
print(modelo_multi.rsquared_adj)    # R² ajustado
```

Para seleccionar el mejor modelo, se recomienda probar distintas combinaciones de predictores y comparar sus métricas. No se debe buscar solo maximizar el R^2, sino también considerar: significancia de los coeficientes (p-value), supuestos cumplidos, interpretabilidad del modelo y sobreajuste. Una técnica frecuente es **stepwise selection**, que agrega o elimina variables según su impacto estadístico.

Además, el uso de información de criterio como el **AIC (Akaike Information Criterion)** o **BIC (Bayesian Information Criterion)** es útil. Estos valores penalizan la complejidad del modelo. Cuanto menor sea el AIC/BIC, mejor es el modelo en términos de equilibrio entre ajuste y simplicidad:

```
print("AIC:", modelo_multi.aic)
print("BIC:", modelo_multi.bic)
```

Finalmente, se recomienda **visualizar el rendimiento comparado** de los modelos mediante un gráfico de barras de R^2 y R^2 ajustado o mediante validación cruzada. Esto permite tomar decisiones más informadas, evitando modelos sobreajustados o poco explicativos. La selección de modelos debe equilibrar precisión, parquedad y robustez.

Test de normalidad del error

Uno de los supuestos fundamentales de los modelos de regresión es que los errores (residuos) del modelo estén distribuidos normalmente. Este supuesto es esencial para que las inferencias estadísticas (como los intervalos de confianza y los tests de significancia) sean válidas. Si los errores no son normales, los estimadores aún pueden ser insesgados, pero las pruebas de hipótesis podrían ser incorrectas.

Existen diversas formas de evaluar la normalidad del error. Una de las más comunes es el gráfico **QQ-Plot (Quantile-Quantile Plot)**, que compara los cuantiles de los residuos con los de una distribución normal. Si los puntos siguen una línea recta, los errores pueden considerarse normales. En Python, se puede hacer con `scipy.stats`:

```
import scipy.stats as stats
import matplotlib.pyplot as plt

stats.probplot(modelo_multi.resid, dist="norm", plot=plt)
plt.title("QQ-Plot de residuos")
plt.show()
```

Además del QQ-Plot, pueden aplicarse pruebas estadísticas como el **test de Shapiro-Wilk**, **Kolmogorov-Smirnov** o **Anderson-Darling**. Estas pruebas devuelven un p-valor que se interpreta como en cualquier test de hipótesis: si es menor que 0.05, se rechaza la normalidad.

```
from scipy.stats import shapiro

stat, p = shapiro(modelo_multi.resid)
print(f"Shapiro-Wilk: Estadístico = {stat:.3f}, p-valor = {p:.3f}")
```

Si el p-valor es menor a 0.05, se debe considerar que los errores no siguen una distribución normal. En ese caso, se pueden aplicar transformaciones sobre la variable dependiente (logaritmo, raíz cuadrada, Box-Cox, etc.) o considerar modelos no lineales más robustos a la no normalidad.

Verificar la normalidad del error no es solo una formalidad técnica: si el modelo será usado para predicción o inferencia (como estimar efectos o calcular intervalos), es crucial asegurar que este supuesto se cumpla. En resumen, el test de normalidad completa el proceso de validación de un modelo lineal, asegurando que los resultados estadísticos derivados del mismo sean confiables.

Cierre ✍️

A lo largo de este manual, exploramos el poder del análisis visual de datos combinado con modelos estadísticos de regresión, centrándonos en las técnicas de regresión lineal simple y múltiple implementadas con Python. Abordamos el cálculo e interpretación de coeficientes, las métricas de error como el MAE, MSE y R^2, y evaluamos la calidad de los modelos mediante criterios estadísticos y visuales, como los gráficos de residuos y QQ-Plots.

También estudiamos los supuestos clave que deben cumplirse para garantizar que los modelos regresivos sean válidos, desde la homocedasticidad hasta la normalidad del error. Aprendimos a aplicar herramientas como Seaborn y Statsmodels no solo para ajustar modelos, sino también para comunicar resultados de forma clara y comprensible a través de visualizaciones efectivas.

Finalmente, profundizamos en la importancia de la selección de variables y del uso del R^2 ajustado, AIC y BIC para construir modelos más robustos. Este conocimiento resulta fundamental para cualquier analista de datos o profesional que desee generar insights válidos a partir de datos reales.

¡Nos vemos de nuevo, más adelante! 👋

Referencias📚

- Scikit-learn. (s.f.). *Supervised learning*. https://scikit-learn.org/stable/supervised_learning.html#supervised-learning
- Statsmodels. (s.f.). *Linear regression models*. https://www.statsmodels.org/stable/regression.html
- Seaborn. (s.f.). *Statistical data visualization*. https://seaborn.pydata.org/
- Brownlee, J. (s.f.). *Linear regression for machine learning*. *Machine Learning Mastery*. https://machinelearningmastery.com/linear-regression-for-machine-learning/
- McKinney, W. (2023). *Python Data Analysis Library (pandas)*. https://pandas.pydata.org

¡Muchas gracias!

Nos vemos en la próxima lección

alkemy